DataShare

« Nous gardons vos fichiers en toute sécurité »

Documentation Technique

v1.0.0

Tables des matières

Architecture de l'application (diagramme) **3**

Choix technologiques **4**

Modèle de données **6**

Documentation d'API **6**

Sécurité et gestion des accès **7**

- Authentification 7
- Gestion des permissions 7
- Sécurisation des données 7
- Limites et protections 8

Qualité, tests et maintenance **9**

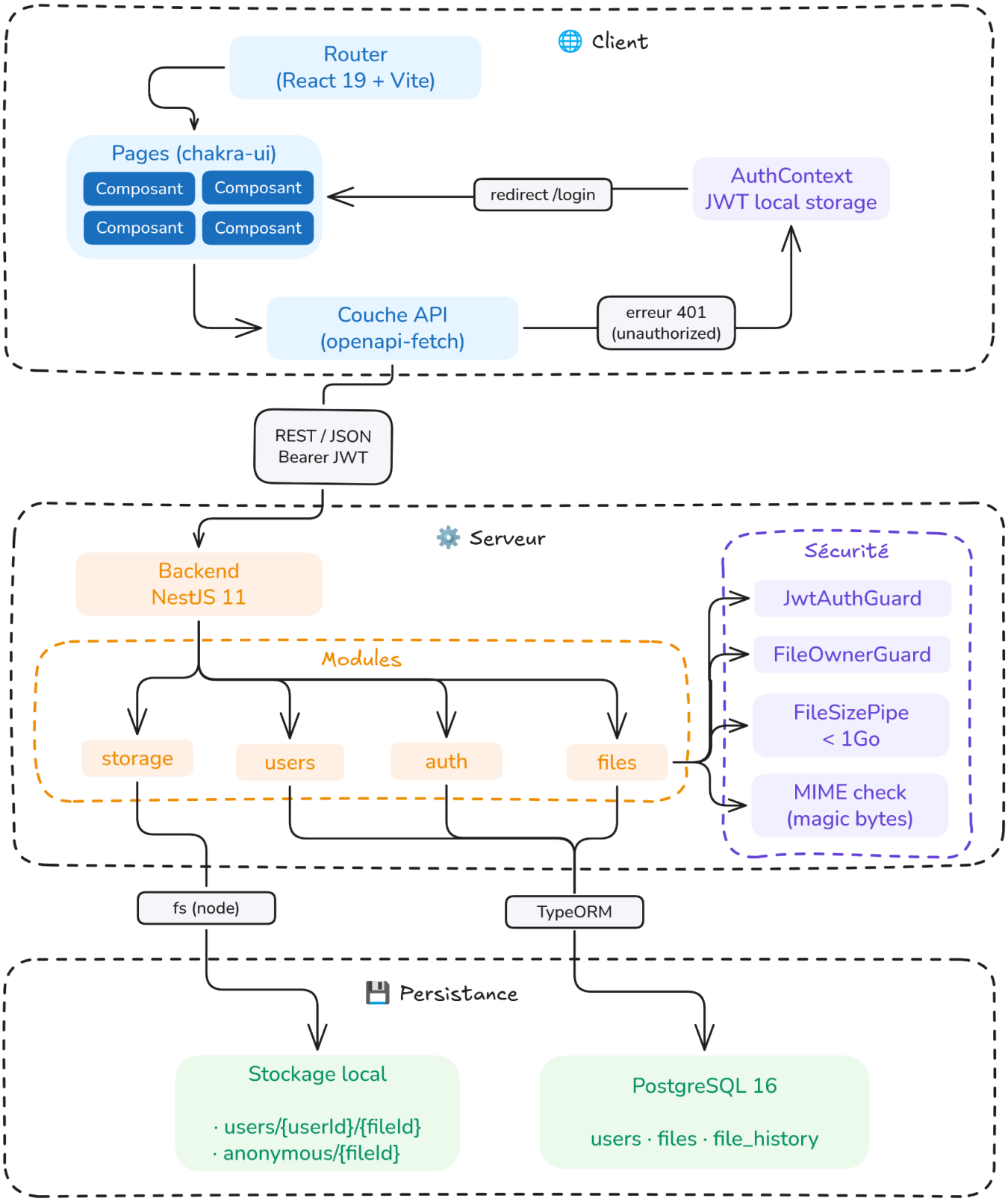
- Qualité 9
- Sécurité 9
- Performance 10
- Maintenance 10

Processus d'installation et d'exécution **11**

Utilisation de l'IA dans le développement **12**

- Outils utilisés 12
- Utilisation de Claude Code 12
- Apports et limites 13

Architecture de l'application (diagramme)

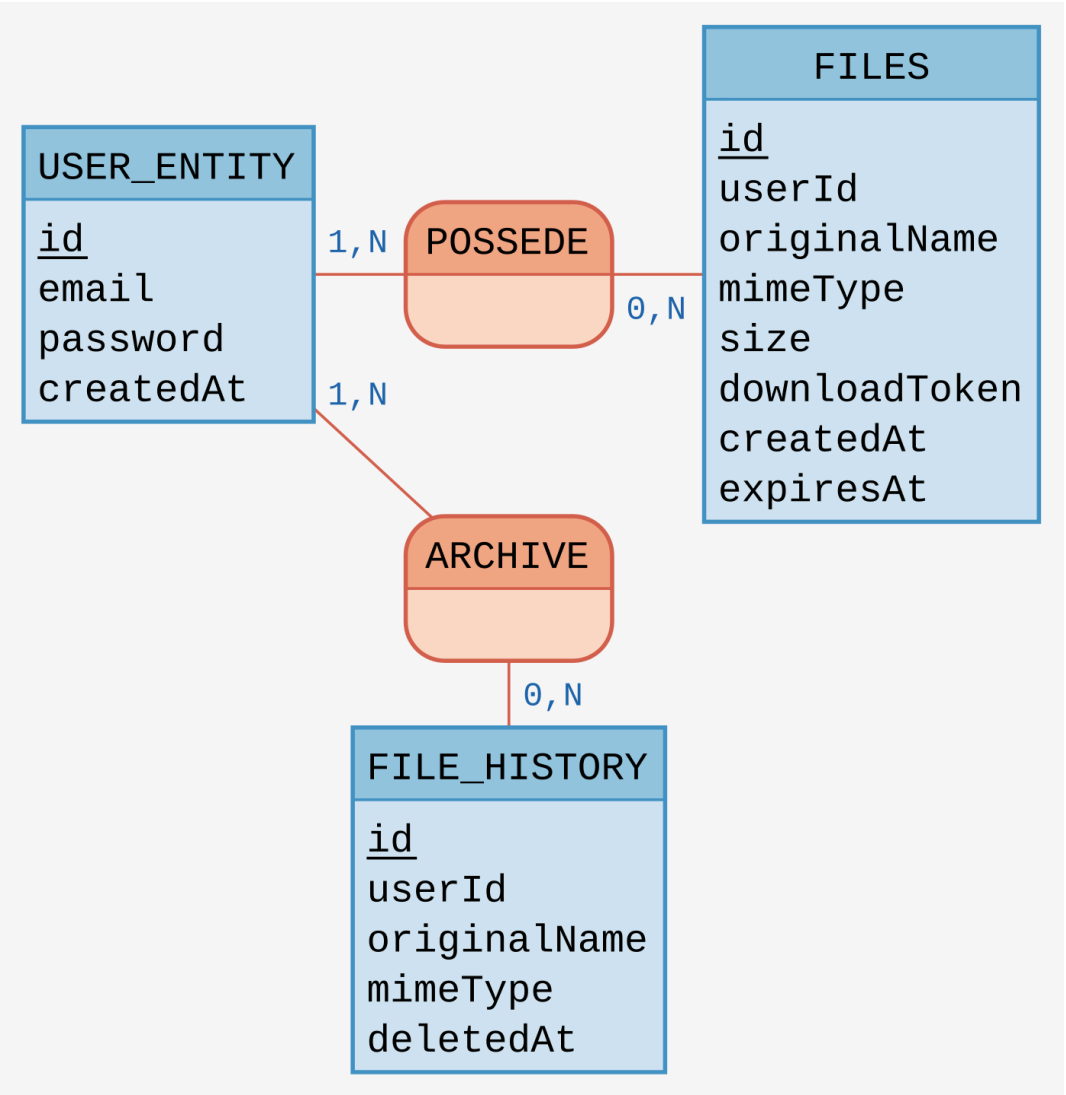


Choix technologiques

Elément	Technologie choisie	Alternatives	Justification
Langage de programmation	Typescript	Java, C#, PHP	- Langage partagé entre le frontend et le backend - Typage statique réduisant le risque d'erreurs
Framework Frontend	React	Angular, VueJS	- Large écosystème et communauté active - Libre choix des librairies tierces (non-opiniated)
Bibliothèque de composants	Chakra-ui v3	Material UI	- Composants accessibles (WCAG) par défaut - Apparence proche du design système demandé (peu de modifications) - Thématisation simple
Framework Backend	NestJS	Spring Boot (Java), NET Core (C#), PHP Symfony/Laravel	- Support natif Typescript - Architecture modulaire et scalable - Middleware et guards pour la sécurité
Type de base de données	SQL	No SQL	- Schéma stable et prévisible, pas besoin de la flexibilité NoSQL - Support ACID garantissant l'intégrité des transactions atomiques (INSERT + DELETE)
Moteur de base de données	PostgreSQL	MongoDB (NoSQL) MySQL, MariaDB	- Base relationnelle robuste et mature - Conformité stricte au standard SQL - Open Source
ORM	TypeORM	Prisma, Sequelize	- Intégration native NestJS - Syntaxe déclarative pour les entités - Migrations automatiques

Elément	Technologie choisie	Alternatives	Justification
Stockage	Système de fichiers local	AWS S3	- Simplicité pour le MVP (pas de configuration cloud) - Coûts nuls pour le développement - Migration facile grâce à l'abstraction prévue dans le code (*storage.service.ts*)
Test backend	Jest	Mocha, Vitest	- Framework par défaut de NestJS : pas de configuration - Mocking intégré - Rapport de couverture intégré
Test frontend	Vitest & Testing Library	Jest	- Optimisé pour Vite - 10 fois plus rapide que Jest pour les tests unitaires - API compatible avec Jest (migration facile)
Test end-to-end	Playwright	Cypress (Typescript), Selenium (Java)	- Support multi-navigateurs - Auto wait intelligent - Exécution en parallèle rapide et gratuite
Outil de versionning	Github	Gitlab	- Plateforme la plus populaire (large communauté, écosystème riche) - Gratuit avec fonctionnalités complètes pour projets privés
Outils IA d'assistance au développement	Claude Code (agent de code) & Github Copilot (auto complétion)	Cursor, Windsurf	- La séparation Copilot (autocomplétion) + Claude Code (agent) offre le meilleur de chaque spécialisation

Modèle de données



Documentation d'API

La documentation OpenAPI est générée par NestJS et accessible sur le serveur backend aux formats suivants:

- Intéractif (Swagger) via l'url */api*
- Schéma json via l'url */api-json*

Ces URLs sont relatives au backend et ne sont donc pas exposées publiquement.

Sécurité et gestion des accès

Authentification

- L'application utilise JWT (JSON Web Token) comme mécanisme d'authentification. Lors de la connexion, le backend génère un token signé avec une clé secrète (*JWT_SECRET*) dont la durée de validité est configurable via *JWT_EXPIRES_IN* (15 minutes par défaut). Ce token doit être transmis dans les en-têtes des requêtes protégées.
- L'identité de l'utilisateur est toujours dérivée du contenu du token JWT : le *userId* n'est jamais lu depuis les paramètres ou le corps de la requête.
- Pour le moment, il n'y a pas de mécanisme de refresh token : à expiration, l'utilisateur doit se reconnecter. Ce mécanisme pourra être implémenté à l'avenir.
- Afin de permettre l'utilisation de certaines routes que l'utilisateur soit ou non authentifié (fonctionnalité d'upload anonyme notamment), un système de token optionnel a été mis en place avec un *OptionalJwtAuthGuard*.

Gestion des permissions

- Il n'y a pas de système de rôles.
- Chaque utilisateur n'a accès qu'à ses propres fichiers.

Sécurisation des données

- Les **mots de passe** sont hachés avec bcrypt (facteur de coût 10) avant persistance en base.
- Les **données sensibles** sont exclues des réponses : le userId n'est jamais retourné dans les réponses API (fichiers et historique), de même que le mot de passe utilisateur, grâce au *ClassSerializerInterceptor* combiné au décorateur @Exclude().
- Protection contre les **attaques timing** : lors de la connexion, une comparaison bcrypt est toujours effectuée même si l'utilisateur n'existe pas (hash factice), afin d'éviter de révéler l'existence d'un compte.

- Le **lien de partage** est basé sur un download_token généré de manière cryptographiquement aléatoire (crypto.randomBytes). L'UUID interne du fichier n'est jamais exposé publiquement.
- **Protection contre l'IDOR** (Insecure Direct Object Reference) : la suppression d'un fichier vérifie non seulement que l'utilisateur est authentifié, mais aussi qu'il est bien propriétaire du fichier demandé (*FileOwnerGuard*). Cela empêche qu'un utilisateur supprime les fichiers d'un autre en devinant un UUID.
- **Validation des entrées** : un ValidationPipe global avec *whitelist: true* filtre tout champ non déclaré dans les DTOs.
- **Non-divulgation des erreurs internes** : le *HttpExceptionFilter* global normalise toutes les erreurs dans un format uniforme. Les erreurs inattendues (500) retournent un message générique: les détails techniques (stack trace, message d'exception brut) ne sont jamais exposés au client.
- Pour **sécuriser les en-têtes HTTP,** le middleware helmet est appliqué globalement.
- La **configuration CORS** n'autorise que le frontend comme origine.
- **HTTPS** : le chiffrement des communications n'est pas géré au niveau de l'application. Il est recommandé de placer le serveur derrière un reverse proxy (nginx, Caddy...) configuré avec TLS en production.

Limites et protections

- La **taille maximale des fichiers** est limitée à 1 Go. Elle est vérifiée par un *FileSizePipe* avant toute écriture sur le disque.
- Le **type MIME** est vérifié côté serveur via les magic bytes du fichier : le Content-Type fourni par le client n'est pas utilisé. Une liste de types autorisée (whitelist) est utilisée au lieu d'une liste de types interdits (blacklist) pour éviter tout oubli.
- **Pagination limitée** : les routes de listing imposent un maximum de 100 éléments par page (limit ≤ 100), afin d'éviter des requêtes trop coûteuses sur la base de données.
- **Expiration des fichiers** : les fichiers expirent entre 1 et 7 jours après l'upload (paramètre expiresIn).

Qualité, tests et maintenance

Qualité

- L'application est testée à plusieurs niveaux:
  - Les tests unitaires backend (Jest) couvrent les cas limites et d'erreur des services métiers (ex: validation MIME, mitigation des timing-attacks à l'authentification)
  - Les tests d'intégration backend (Jest + PostgreSQL) couvrent tous les endpoints
  - Les tests unitaires frontend (Vitest + Testing Library) couvrent la logique métier implémentée, sans tester le framework en lui-même
  - Les tests de bout en bout (Playwright) couvrent les fonctionnalités obligatoires du MVP
- L'objectif de couverture de tests était de 70%. La couverture atteinte est:
  - Backend: 94% statements / 75% branches ✅
  - Frontend: 83% branches ✅
- Tous les tests sont au vert.

Sécurité

- Résultat du scan de vulnérabilités (npm audit):
  - 0 critique
  - 0 haute
  - 8 modérées
- La vulnérabilité causée par une dépendance directe a été corrigée suite au scan en installant une version plus récente patchée.
- Les autres vulnérabilités sont acceptables car elles proviennent de dépendances de développement uniquement, non incluses dans le build de production.

Performance

Le test de **performance backend** a été effectué avec k6 sur l'endpoint critique *POST /files* (upload anonyme). Le scénario simule une montée en charge de 0 à 10 virtual users sur 60 secondes.
En conditions locales, la performance est excellente. Le test ayant été effectué avec un fichier de 54 KB, il serait pertinent de rejouer le scénario avec des fichiers proches de la limite de 1 Go acceptée par l'application.

La **performance frontend** a été analysée avec Lighthouse sur le build de production. Le score de performance est bon mais le chargement initial dépasse les cibles recommandées.
Le principal levier d'amélioration est la réduction de la taille du chunk chakra UI en suivant le guide officiel d'optimisation de la bibliothèque de composants. L'optimisation du chargement des polices est également une piste.

Maintenance

- Les mises à jour de dépendances npm sont gérées en partie automatiquement via Dependabot, qui ouvre des pull requests pour les correctifs de sécurité et les mises à jour patch/minor. Les mises à jour majeures restent manuelles et nécessitent une revue approfondie avant application.
- Un audit de sécurité (npm audit) est prévu hebdomadairement, les mises à jour patch/minor mensuellement, et les mises à jour majeures trimestriellement.
- Une checklist de maintenance mensuelle est définie : elle couvre l'audit des dépendances, l'application des mises à jour, l'exécution de tous les niveaux de tests, la vérification du build et de l'espace disque de stockage.
- Les principaux risques identifiés sont liés aux dépendances de sécurité critiques (bcrypt, @nest/jwt, file-type) et aux montées de version majeures des frameworks (NestJS, React, TypeORM), qui nécessitent une revue manuelle et une exécution des tests avant déploiement.

Processus d’installation et d’exécution

Prérequis :

- Node.js >= 20
- npm >= 10
- Docker et Docker Compose.

Installation :

```
git clone https://github.com/manon-hubert/OCP3-datashare.git
cd OCP3-datashare
npm install
```

Configuration des variables d’environnement :

Trois fichiers .env sont à créer à partir des exemples fournis (.env.example) :

- à la racine
- dans backend/
- dans frontend/

Lancement de l’application:

```
npm run db:start # démarre PostgreSQL via Docker
npm run dev      # lance le backend et le frontend en parallèle
```

Utilisation de l’IA dans le développement

Outils utilisés

Le développement de ce projet a été effectué à l'aide de deux outils d'intelligence artificielle, répondant à deux besoins distincts :

- **Claude Code** (agent de code)
- **GitHub Copilot** (autocomplétion inline)

L'autocomplétion par GitHub Copilot ne nécessite pas de configuration particulière et ne propose que des suggestions de code. Son utilisation ne nécessite pas de posture particulière et ne permet pas de déléguer des tâches. L'assistant Copilot Chat n'a pas été utilisé pour ce projet.

Utilisation de Claude Code

Claude Code est un agent de code autonome. Afin d'optimiser son utilisation, un fichier d'instructions *CLAUDE.md* a été créé lors de l'initialisation du projet et contient toutes les informations nécessaires sur le contexte fonctionnel et technique. Ces instructions sont disponibles sur GitHub et réutilisables par tout développeur du projet DataShare souhaitant utiliser cet agent.

Plusieurs utilisations de Claude Code sont à distinguer, relevant de deux postures différentes :

Assignation de tâche :

La User Story *"Consultation d'historique"* a été attribuée à Claude avec des instructions basiques et une capture d'écran Figma. Le cadrage aurait pu être amélioré avec des instructions plus précises, par exemple pour éviter le développement de fonctionnalités hors scope (ex. : bouton "Déconnexion" visible sur la maquette mais pas encore implémenté). Des correctifs manuels ont ensuite été apportés, en particulier sur le style. Ce processus est tracé dans l’historique github.

Binômage :

- **Assistance technique** : Claude Code a été utilisé comme assistant de débogage et de recherche documentaire. Il a été connecté au serveur MCP Chakra UI pour faciliter la recherche d'informations sur certains composants et le theming.
- **Assistance à la documentation** : Claude Code a facilité la rédaction des fichiers Markdown, notamment pour la synthèse du plan de test et la mise en forme des résultats de tests de performance à partir des sorties brutes.
- **Revue de code** : la commande /review de Claude Code permet d'obtenir des suggestions d'amélioration sur des fichiers, jouant un rôle similaire à celui d'un relecteur dans une équipe.

Apports et limites

Apports :

L'utilisation d'un agent de code autonome est un gain de temps sur les tâches bien délimitées. L'accélération de la recherche documentaire via l'intégration MCP est un gain appréciable, tout comme la production rapide de livrables de documentation. En solo, l'utilisation de la commande /review compense partiellement l'absence de relecteur humain.

Limites :

La qualité des résultats est très dépendante du cadrage initial: un brief imprécis entraîne facilement du hors scope. Une relecture et des corrections manuelles restent nécessaires lors de l'assignation de tâche, ce qui s'approche de la posture à adopter avec un développeur junior.

Le *CLAUDE.md* atténue les problèmes de cohérence avec les conventions du projet, mais ne couvre pas les décisions techniques non documentées.